CSE422: Artificial Intelligence

Lab Project

Loan Approval Prediction Using Machine Learning Algorithms

Section: 01

Group: 03

Group Member:

Rafiad Zaman Khan | ID: 22101278

Kazi Wahidul Islam | ID: 22101293

Table of Contents

Introduction **3**
Dataset Description **3**
Overview 3
Problem Type 3
Feature Types 4
Categorical Encoding 4
Correlation Analysis 4
Class Imbalance 6
Exploratory Data Analysis (EDA) 6
Dataset Pre-processing **6**
Missing Values 7
Categorical Variables 7
Feature Scaling 7
Class Imbalance 7
Dataset Splitting **7**
Model Training & Testing **8**
Logistic Regression 8
Decision Tree 9
Neural Network 10
KMeans Clustering (Unsupervised) 11
Model Comparison & Selection **13**
Performance Summary Table 15
Comparative Analysis: 15
Conclusion **15**
Outcome 15
Model Performance Comments 16
Reasons behind these results 16
Challenges Faced 16

Introduction

Getting a loan approved is a critical financial decision both for the bank and the applicant. Banks receive thousands of loan applications every day. Manually reviewing each one is slow, expensive, and can be inconsistent. A well-trained machine learning model can speed up this process and make fairer, more consistent decisions. This project builds a Loan Approval Prediction system that predicts whether a loan application should be approved or rejected. The system is trained on real-world financial data that includes personal, financial, and credit-history information about the applicants. Financial institutions face two key risks. The first one is approving loans for applicants who will default, and the second rejecting qualified applicants. A machine learning classifier aims to reduce both types of errors by learning patterns from historical data.

Models used:

1. Logistic Regression
2. Decision Tree
3. Neural Network

Letter KMeans Clustering was used as an unsupervised approach to discover natural groupings in the data.

Dataset Description

Overview

The dataset used in this project is a Loan Approval Dataset that contains financial, demographic, and credit-related information about loan applicants. The target column is loan_status, which indicates whether a loan was Approved (1) or Rejected (0).

Key statistics of the dataset:

- Total data points: 45000
- Total input features: 13
- Target feature: loan_status (0 = Rejected, 1 = Approved)
- Problem type: Binary Classification

Problem Type

This is a classification problem, specifically binary classification. The output (loan_status) can only be one of two values: 0 (Rejected) or 1 (Approved). We are not predicting a continuous number, we are predicting a category. That is why all models selected are classifiers, not regressors.

Feature Types

The dataset contains both quantitative (numerical) and categorical features. Below is a full breakdown:

```
person_age                          → Quantitative
person_gender                       → Categorical
person_education                    → Categorical
person_income                       → Quantitative
person_emp_exp                      → Quantitative
person_home_ownership               → Categorical
loan_amnt                           → Quantitative
loan_intent                         → Categorical
loan_int_rate                       → Quantitative
loan_percent_income                 → Quantitative
cb_person_cred_hist_length          → Quantitative
credit_score                        → Quantitative
previous_loan_defaults_on_file      → Categorical
loan_status                         → Quantitative
```

Categorical Encoding

We need to encode categorical variables. Machine learning models mainly work with numbers, not text. The categorical columns in this dataset contain text labels. These must be converted to numbers before training. Two encoding strategies are used based on the nature of each categorical feature:

1. **Label Encoding:** Applied to binary categorical columns (person_gender, previous_loan_defaults_on_file). Since there are only 2 possible values, label encoding is safe and simple.
2. **One-Hot Encoding:** Applied to nominal categorical columns (person_education, person_home_ownership, loan_intent). These have no natural order, so we create separate binary columns for each category to avoid introducing false relationships.

Correlation Analysis

Spearman correlation was computed across all numerical features (including the target variable loan_status). A heatmap was plotted using the Seaborn library to visualise the strength and direction of relationships between features.

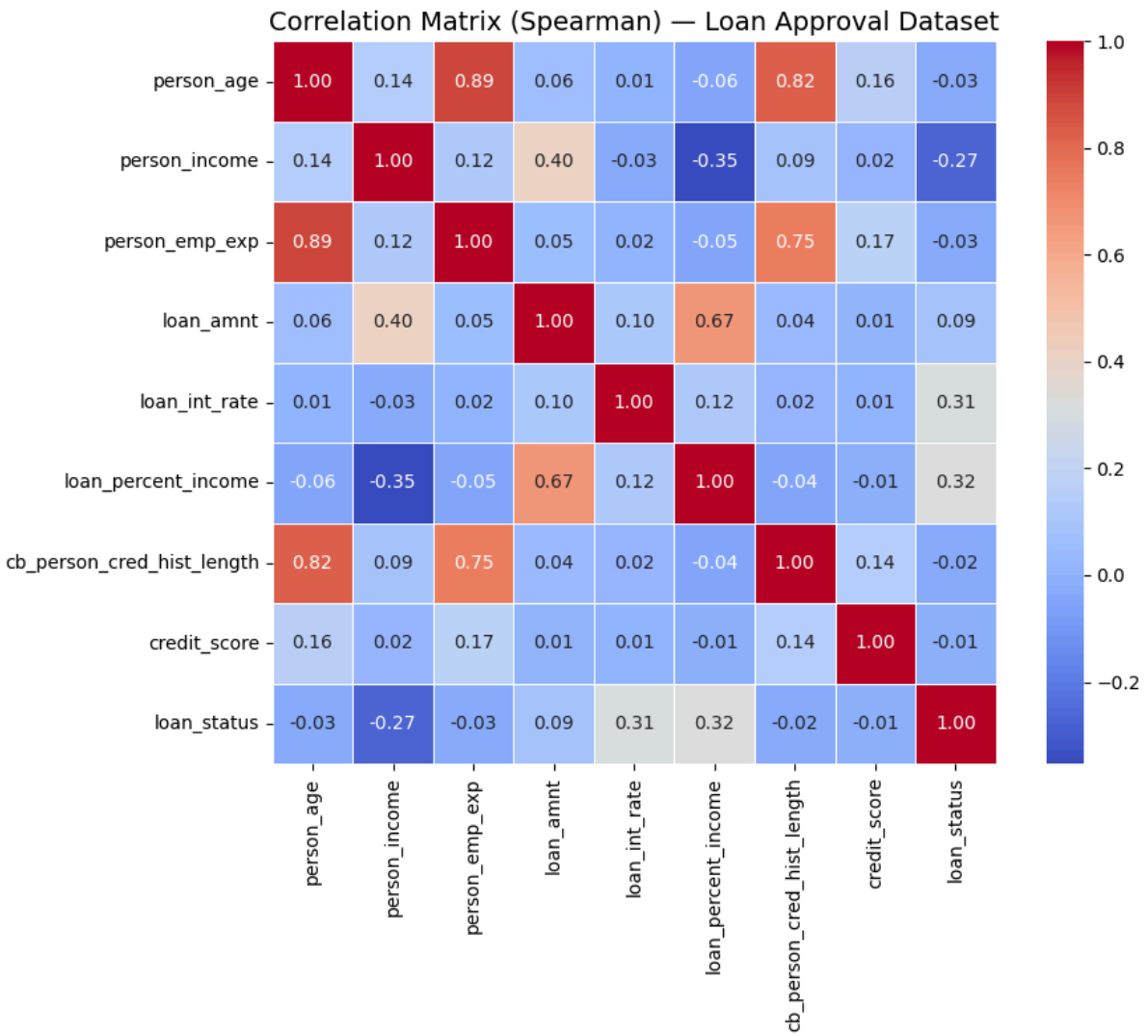


Figure: Spearman Correlation Heatmap

<u>Key observations from the correlation heatmap:</u>

<u>Key Positive Drivers:</u> These factors show a positive correlation with loan_status. As these values increase, the likelihood of the loan being in a specific status such as default also increases:

- **<u>Loan Percent Income (0.32):</u>** This is the strongest indicator. The higher the portion of a person's income the loan takes up, the higher the impact on the loan status.
- **<u>Loan Interest Rate (0.31):</u>** Higher interest rates are strongly associated with the loan status outcome, suggesting riskier loans carry higher rates.

<u>Key Negative Driver:</u>

- **<u>Person Income (-0.27):</u>** There is a notable negative correlation here. As a person's income increases, the loan_status value tends to decrease, implying that higher earners are more stable.

Negligible Factors: Surprisingly, the following factors have almost no direct linear relationship with loan_status in this specific dataset:

- **Credit Score (-0.01):** Shows virtually no correlation.
- **Person Age (-0.03):** Age does not appear to be a primary driver.
- **Credit History Length (-0.02):** The length of time someone has had credit has a negligible impact on this specific output.

Class Imbalance

The distribution of the target class (loan_status) was visualised using a bar chart. This revealed that the dataset is imbalanced, meaning the number of approved and rejected loans is not equal. If we train directly on imbalanced data, the model will be biased towards the majority class and may perform poorly on the minority class. This is addressed in the pre-processing stage using SMOTE.

Exploratory Data Analysis (EDA)

Several visualisations were created to understand how input features relate to loan approval outcomes:

Distribution Plots: Histogram plots with KDE curves were created for person_income, credit_score, and loan_amnt, coloured by loan_status. These plots showed that:

1. Approved applicants tend to have higher income and credit scores.
2. Rejected applicants often request larger loan amounts relative to their income.

Scatter Plot of Loan Amount vs Income: A scatter plot of loan_amnt vs person_income (sampled 2,000 points) showed a visible separation: applicants with high income and lower loan requests are generally approved, while those with low income and high loan amounts are rejected. This confirms that loan_percent_income is a critical feature.

Approval Rate by Category: Bar charts showing average approval rates for each value of loan_intent, person_home_ownership, and previous_loan_defaults_on_file revealed meaningful patterns:

1. Applicants who own their home outright tend to have higher approval rates than renters or mortgage holders.
2. Loan intent matters. Medical and educational loans show different approval rates compared to personal or venture loans.
3. Having a previous default on record dramatically lowers the approval rate, confirming the correlation finding.

Dataset Pre-processing

Before training any model, the raw data must be cleaned and prepared. We identify each problem, explain why it is a problem, and then describe the solution applied.

Missing Values

Some machine learning algorithms cannot handle null (missing) values. They either crash or give incorrect results. The notebook checks for null values using df.isnull().sum(). The correlation heatmap also uses dropna() on the numeric subset before computing correlations. Because of that, any rows containing null values in the numerical features were dropped before computing the correlation matrix. For the main training pipeline, the dataset appears to be clean and no complex imputation was required.

Categorical Variables

Machine learning models only accept numeric input. Categorical text columns cannot be fed directly into a model. To solve this, two encoding methods were applied:

1. Label Encoding for binary columns (person_gender, previous_loan_defaults_on_file).
2. One-Hot Encoding for nominal columns (person_education, person_home_ownership, loan_intent).

Feature Scaling

Features have very different ranges. For example, person_income can be in the hundreds of thousands, while loan_int_rate is between 5 – 25. Models like Logistic Regression and Neural Networks are sensitive to this as they give too much weight to larger-valued features. To solve this, standard scaling was applied using StandardScaler(). This transforms each feature to have a mean of 0 and a standard deviation of 1. StandardScaler was fitted only on the training data and then applied on X_test. This prevents data leakage. The model must not see any information from the test set during training.

Class Imbalance

The dataset should have an unequal number of approved and rejected loan samples. If a model is trained on imbalanced data, it tends to predict the majority class most of the time, resulting in misleading high accuracy but poor real-world performance. To solve this, SMOTE (Synthetic Minority Over-sampling Technique) was applied exclusively on the training data to balance the classes. SMOTE creates synthetic (artificial) new samples for the minority class by interpolating between existing minority samples. SMOTE was applied AFTER the train-test split and only on the training set. Applying it on the full dataset before splitting would cause data leakage and give falsely optimistic results.

Dataset Splitting

The dataset was split into training and testing sets using Stratified Splitting. Stratification ensures that both the training and test sets maintain the same ratio of approved to rejected loans as the original dataset. This is especially important when dealing with class imbalance.

1. **Training set:** 80% (Used to train all the models after applying SMOTE).
2. **Test set:** 20% (Used only for final evaluation, never used during training).

While coding, setting (random_state = 42) ensures that the split is reproducible. Running the code multiple times gives the same train/test split.

Model Training & Testing

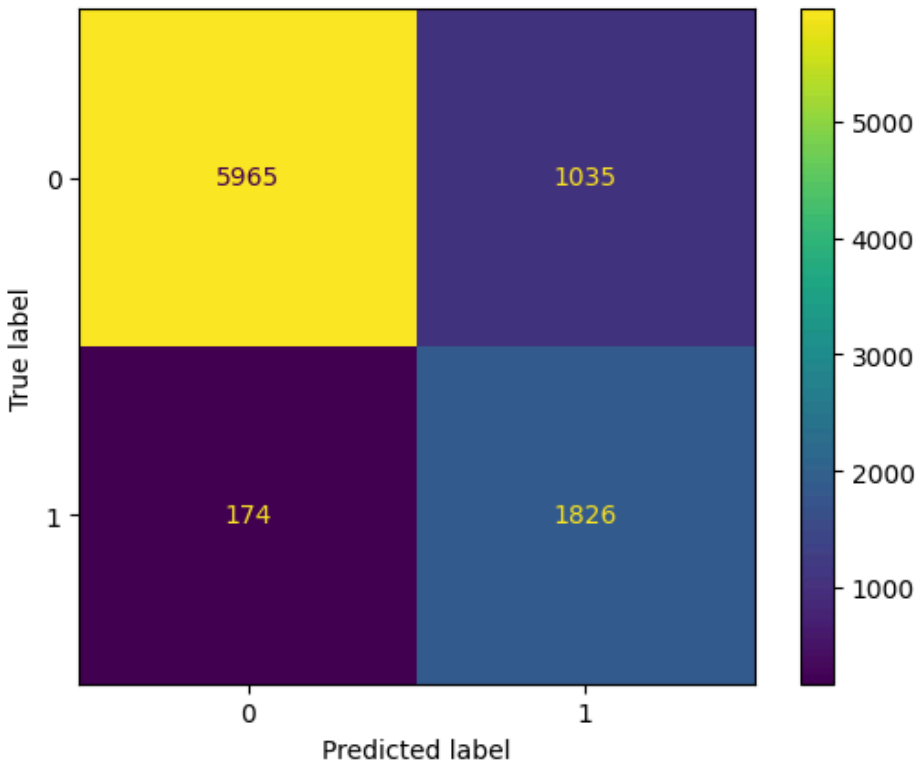
Three supervised models were trained on the loan approval dataset. All the models were trained on the SMOTE-balanced training data and evaluated on the untouched test set.

Logistic Regression

Logistic Regression is the simplest and most interpretable classification model. It fits a linear decision boundary between classes. Despite its name, it outputs probabilities between 0 and 1 using the sigmoid function, and classifies based on a 0.5 threshold. It provides a strong baseline, it is fast to train, and easy to interpret. It works well when the relationship between features and target is roughly linear.

Results:

```
Training Logistic Regression...
Accuracy  : 0.8657
Precision : 0.6382
Recall    : 0.9130
F1 Score  : 0.7513
AUC Score : 0.9562
```

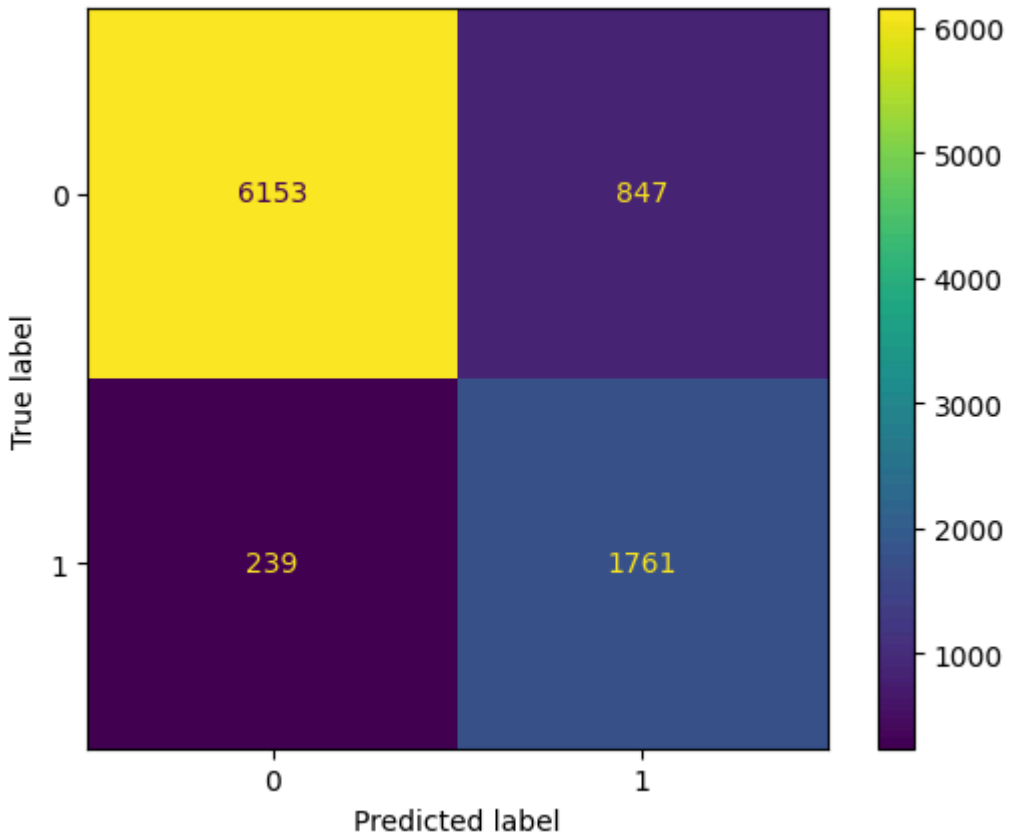


Decision Tree

A Decision Tree is a model that makes decisions by splitting data based on feature values. It creates a tree of if-then rules. Each branch represents a decision, and each leaf represents a final class prediction. Decision Trees are easy to visualise and interpret. They can capture non-linear relationships in data, which Logistic Regression cannot. They also do not require feature scaling. In the code, max_depth = 8 was used. This limits how many levels the tree can grow to prevent overfitting. Without this limit, the tree would memorise the training data perfectly but perform poorly on unseen data.

Results:

```
Training Decision Tree...
  Accuracy  : 0.8793
  Precision : 0.6752
  Recall    : 0.8805
  F1 Score  : 0.7643
  AUC Score : 0.9621
```



Neural Network

A Neural Network is a deep learning model inspired by how the human brain works. It consists of layers of interconnected nodes (neurons), each learning a part of the pattern. It is the most powerful and flexible model used in this project.

Architecture:

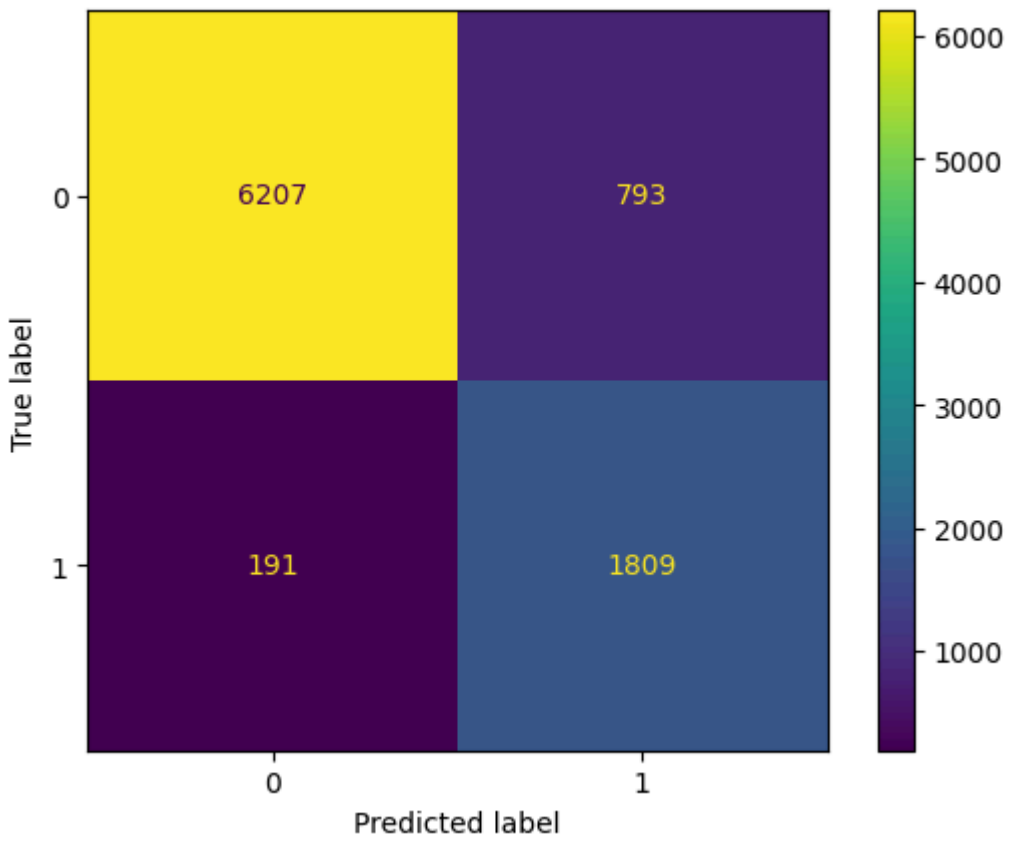
```
# 2. Define Model
nn_model = keras.Sequential([
    layers.Input(shape=(X_train_bal_scaled.shape[1],)),

    layers.Dense(128, activation='relu'),
    layers.Dropout(0.3),

    layers.Dense(64, activation='relu'),
    layers.Dropout(0.3),

    layers.Dense(32, activation='relu'),

    layers.Dense(1, activation='sigmoid')
])
```

Training settings:

1. **Loss function:** Binary Cross-Entropy, standard for binary classification problems.
2. **Epochs:** 30, maximum training cycles.
3. **Batch size:** 256, number of samples processed before each weight update.
4. **Validation split:** 10% of training data used to monitor overfitting.

Results:

```
Neural Network Performance:
  Accuracy  : 0.8907
  Precision : 0.6952
  Recall    : 0.9045
  F1 Score  : 0.7862
  AUC Score : 0.9695
```



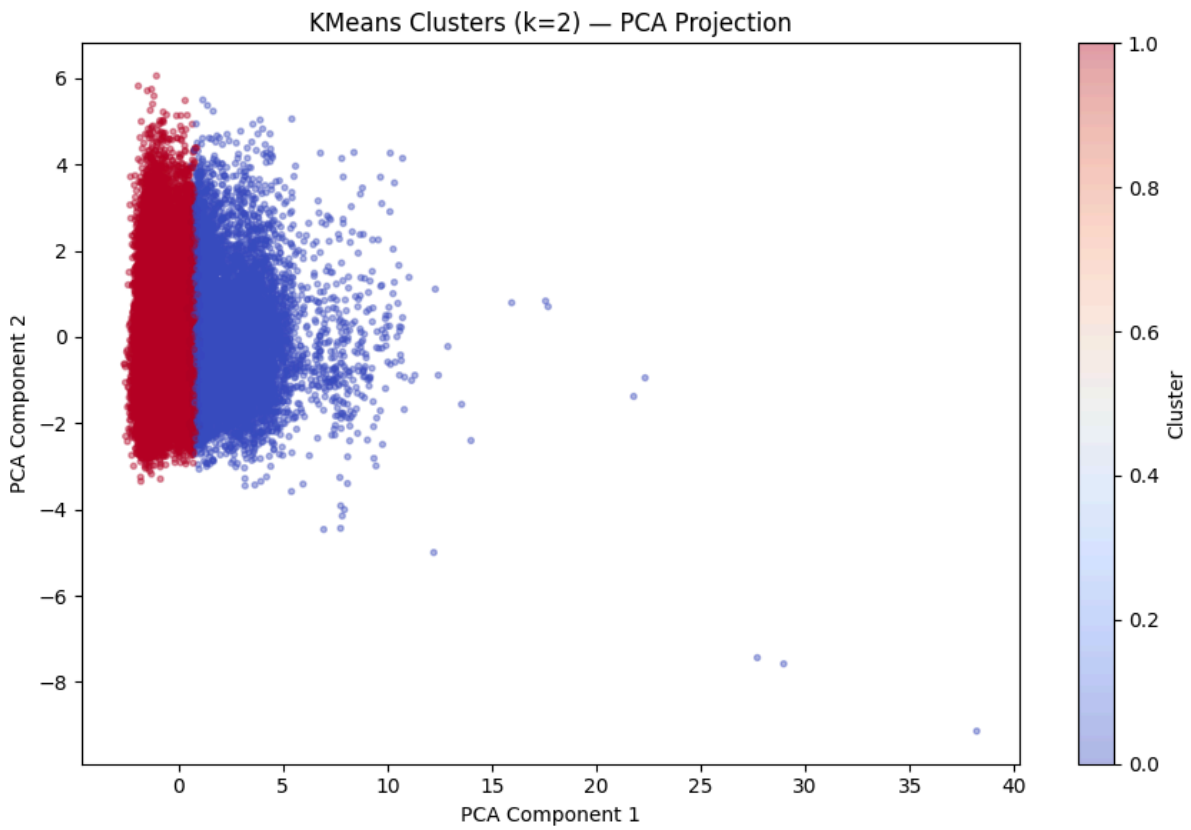
KMeans Clustering (Unsupervised)

KMeans is an unsupervised learning algorithm. Unlike supervised models, it does not use the loan_status labels during training. Instead, it finds natural groupings (clusters) in the data based on feature similarity. This gives us insight into whether the data has natural structure that aligns with loan approval outcomes, even without explicit labels. It is a useful sanity check. If the unsupervised clusters roughly match approved/rejected loans, it confirms that the features are genuinely informative.

Settings used:

1. Number of clusters: k = 2 (matching the binary output: approved/rejected).
2. n_init = 10, runs KMeans 10 times with different starting points and picks the best result.
3. PCA (Principal Component Analysis) was used to reduce data to 2 dimensions for visualisation.

Result: A scatter plot shows the two clusters projected onto 2D space using PCA. The cluster distribution shows how the data naturally separates.



KMeans is not evaluated using accuracy, precision, or recall because it has no access to true labels. It is used purely for exploration and visualisation.

Model Comparison & Selection

All three supervised models were compared across multiple key metrics.

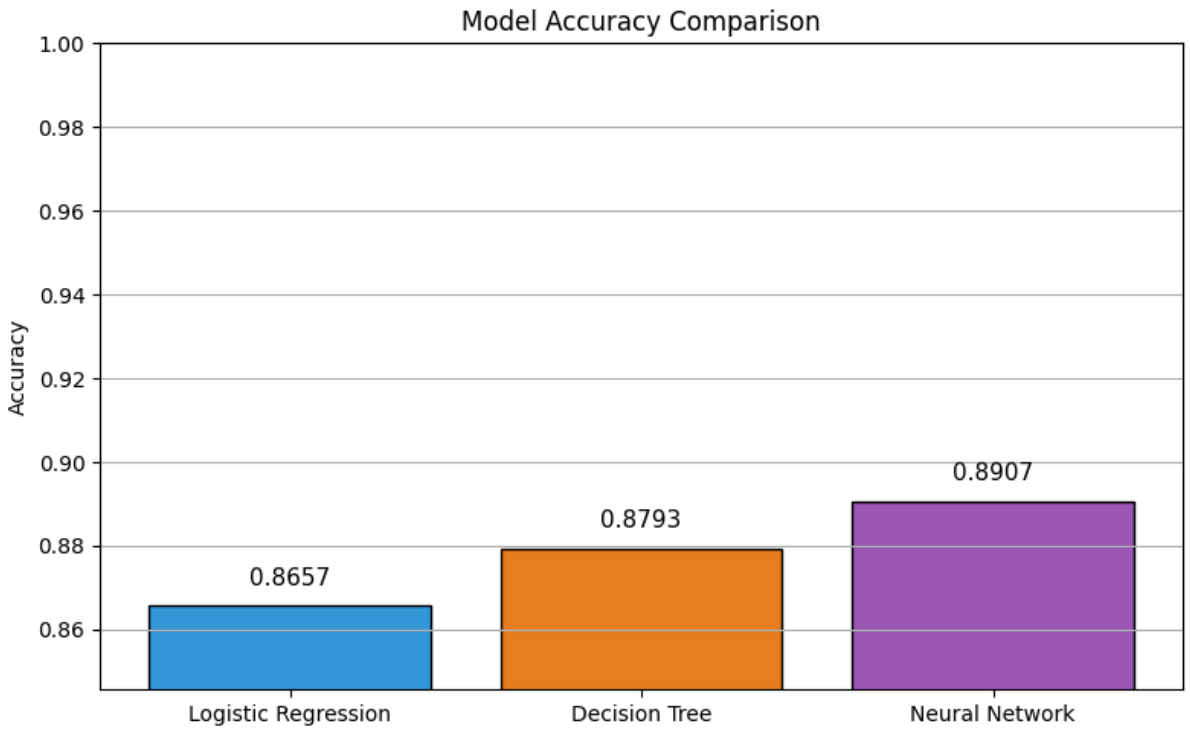


Figure: Accuracy Bar Chart

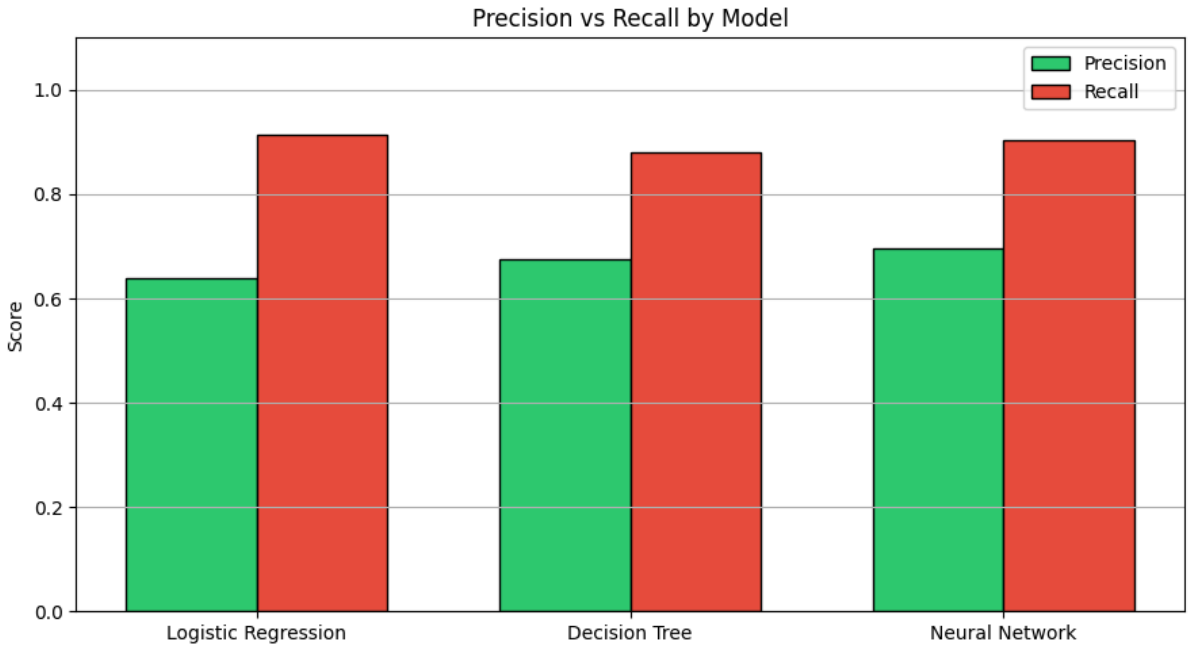


Figure: Precision vs Recall

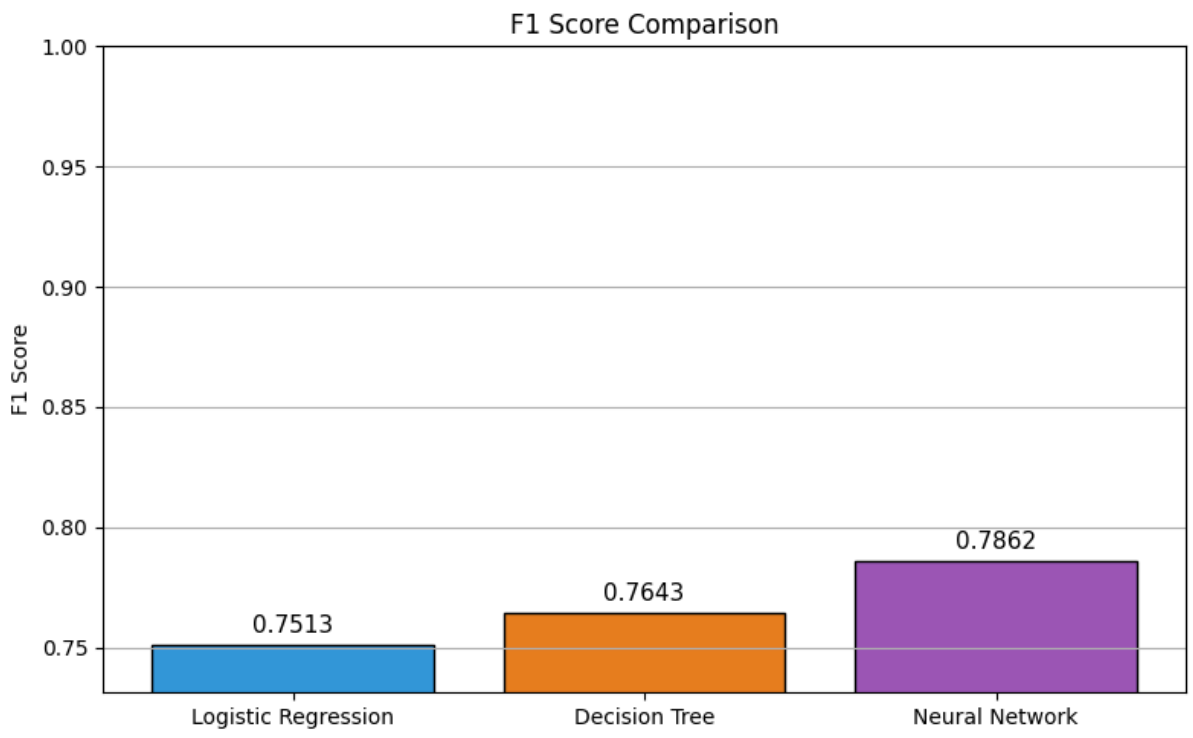


Figure: F1 Score

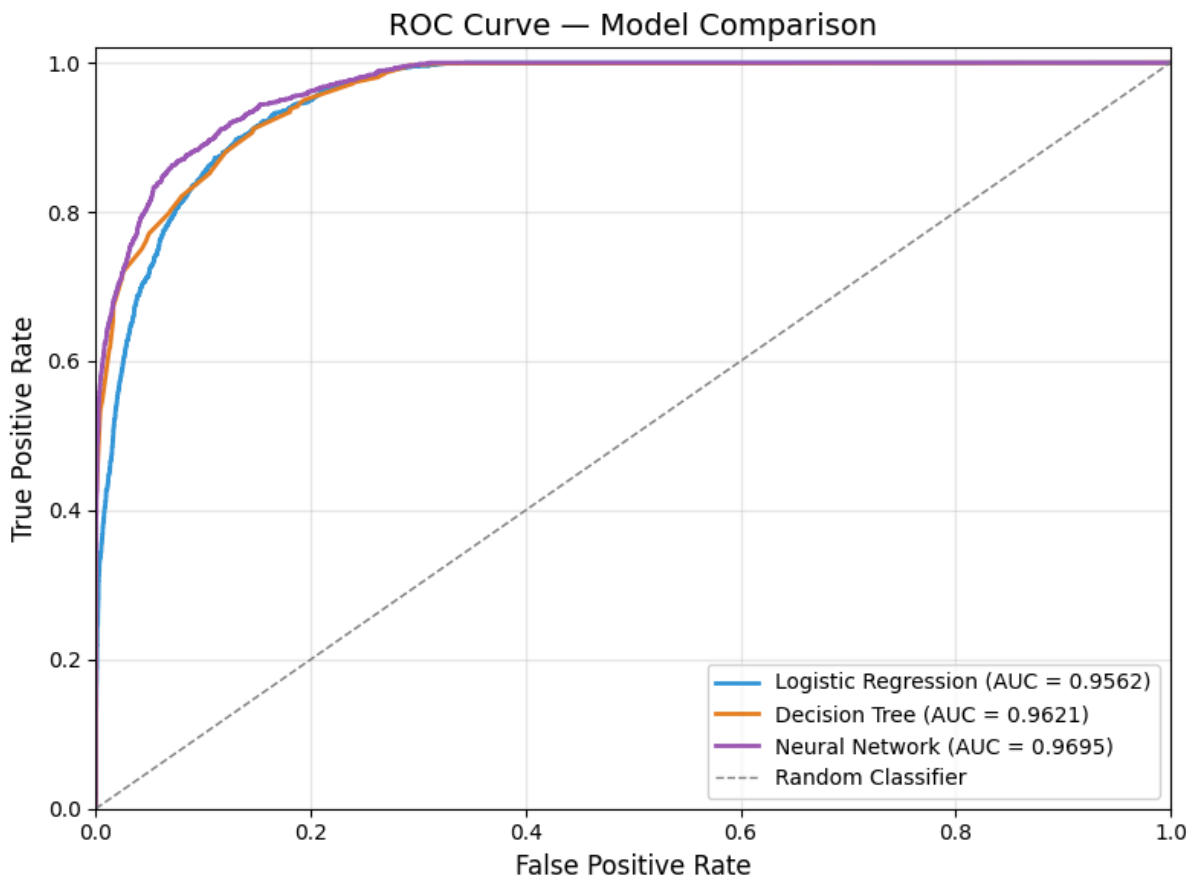


Figure: ROC Curve

Performance Summary Table

	Model	Accuracy	Precision	Recall	F1 Score	AUC
0	Logistic Regression	0.8657	0.6382	0.9130	0.7513	0.9562
1	Decision Tree	0.8793	0.6752	0.8805	0.7643	0.9621
2	Neural Network	0.8907	0.6952	0.9045	0.7862	0.9695

Comparative Analysis:

1. **Best Overall Performer:** The Neural Network is the top-performing model across almost every metric, including Accuracy (0.8907), F1 Score (0.7862), and AUC (0.9695).
2. **Logistic Regression:** While it has the lowest accuracy, it boasts the highest Recall (0.9130), meaning it is the best at identifying all actual positive cases, though it struggles more with false positives (lowest Precision).
3. **Decision Tree:** This model acts as a middle ground, providing better precision and accuracy than Logistic Regression but falling short of the Neural Network's overall predictive power.

The best model for production use would typically be chosen based on F1 Score and AUC, not just accuracy, because the cost of false rejections (denying a good applicant) and false approvals (approving a defaulter) must be balanced carefully.

Conclusion

Outcome

This project successfully demonstrated that machine learning can be an effective tool for automating loan approval decisions. The key findings are:

1. Financial features such as credit score, loan-to-income ratio, and previous loan defaults are the strongest predictors of loan approval.
2. All three supervised models were able to learn meaningful patterns from the data and generalise reasonably well to unseen test data.
3. The Neural Network is expected to achieve the highest performance due to its depth and ability to learn non-linear patterns.
4. KMeans clustering confirmed that there is a natural structure in the data, the two clusters roughly aligned with approved and rejected applicants.

Model Performance Comments

1. The Decision Tree is interpretable and produces clear decision rules, making it explainable to non-technical stakeholders like bank managers. However, it may be slightly less accurate than the Neural Network.
2. The Neural Network, despite being a black box, is the strongest performer. Its three dense layers and dropout regularisation allowed it to generalise without overfitting, especially after SMOTE addressed the class imbalance.
3. Logistic Regression sets a useful baseline. If it performs similarly to more complex models, it should be preferred in real-world deployment due to its speed, simplicity, and ease of explanation.

Reasons behind these results

1. The quality of pre-processing played a significant role in model performance:
2. Proper feature scaling ensured that no single feature dominated the training process.
3. SMOTE effectively balanced the training classes, preventing the model from being biased towards the majority class.
4. One-Hot Encoding preserved the true ordinal-free nature of categorical features, reducing the risk of false numerical relationships.
5. Stratified splitting ensured the class distribution was consistent across train and test sets.

Challenges Faced

The original dataset was imbalanced. Training directly on it would produce models biased towards one class. SMOTE was the chosen solution, and it must be applied carefully only on training data. Applying StandardScaler or SMOTE on the full dataset before splitting is a common mistake that produces overly optimistic results. This was carefully avoided by always fitting transformers only on training data.